Design and Evaluation of Machine Learning Frameworks for Proportional Fairness Power and Resource Allocation in Multi-User Wireless Networks

Sai Rithvik Achutuni

May 6, 2026

Challenge in next gen networks



Modern network architectures are mandated to accommodate an exponentially growing density of heterogeneous devices while delivering ultra-reliable, low-latency connectivity.

The performance benchmarks established for 6G target peak data rates of 1.0 terabits per second and air-interface latencies approaching 0.1 milliseconds, figures that vastly exceed the capabilities of legacy cellular infrastructures

The core problem

Dynamically allocating limited radio resources—specifically, an allocation across K shared orthogonal frequency resources and a finite total transmit power budget P —among N competing client devices in a multi-user environment.

The problem of choosing transmit power dynamically in response to physical channel conditions in order to maximize the downlink sum-rate with maximal power constraints is NP-hard.[1]

Limits of Traditional Methods

- computational complexity
- traditional models fail to account for hardware and channel imperfections
- specific hardware components such as low-resolution A/D, nonlinear amplifier are challenging to be developed. [2]

System Model

- The environment consists of a base station serving N active clients.
- Clients compete for K orthogonal frequency resources.
- $\mathbf{P}_{n,k}$: Continuous transmit power allocated to client n on resource k.
- **Sum-Power Constraint:** Total aggregate power is limited by a physical budget P_{total}.

$$\sum_{n=1}^{N} \sum_{k=1}^{K} P_{n,k} \leq P_{total}$$

The Wireless Environment: Interference & Capacity

- **SINR Formulation:** The Signal-to-Interference-plus-Noise Ratio ($\gamma_{n,k}$) for client n on resource k accounts for co-channel interference:

$$\gamma_{n,k} = \frac{p_{n,k}|h_{n,k}|^2}{\sum_{j \neq n} p_{j,k}|h_{n,j}|^2 + \sigma^2}$$

- **Achievable Data Rate:** According to the Shannon-Hartley theorem, the data rate $r_{n,k}$ given bandwidth B is:

$$r_{n,k} = B \log_2(1 + \gamma_{n,k})$$

- **Aggregate Throughput:** The instantaneous total rate for client n is $R_n = \sum_{k=1}^{K} r_{n,k}$.

The Trade-Off: Throughput vs. Fairness

- **The Core Conflict:** Designing a scheduler to maximize total network throughput (sum-rate) inherently favors users with optimal, unobstructed channel conditions.
- **Cell-Edge Starvation:** Exclusively prioritizing these strong channels leads to the severe resource deprivation or "starvation" of cell-edge users who experience high path loss and interference.
- **Enforcing Equity:** Maintaining fairness requires the network to actively allocate finite resources to users with poorer channel conditions, guaranteeing a baseline quality of service for everyone.

The α-Fairness Utility Paradigm

- **Objective:** Maximize overarching network utility $U_\alpha(\mathbf{R})$ to navigate the throughput versus fairness trade-off.
- **Mathematical Framework:**

$$U_\alpha(\mathbf{R}) = \begin{cases} \sum_{n=1}^{N} \frac{R_n^{1-\alpha}}{1-\alpha}, & \text{if } \alpha \geq 0, \alpha \neq 1 \\ \sum_{n=1}^{N} \log(R_n), & \text{if } \alpha = 1 \end{cases}$$

- **Operational Behaviors:**
  - $\alpha = 0$: Sum-Rate Maximization (favors strong channels, risks edge-user starvation).
  - $\alpha = 1$: Proportional Fairness (maximizes geometric mean of user rates).
  - $\alpha \to \infty$: Max-Min Fairness (guarantees absolute equity but severely penalizes capacity).

Literature Review

Deep Unrolling

- **Core Concept:** Deep unrolling converts classical iterative optimization algorithms into deep neural networks by mapping each iteration directly to a network layer.[3]
- **Mathematical Rigor:** This approach blends the structural insights of proven iterative algorithms with the adaptive learning capabilities of neural networks, offering superior interpretability over generic models.
- **Constraint Satisfaction:** Frameworks can utilize projection gradient descent during forward passes to strictly enforce complex, non-convex constraints, such as total power budgets.

Deep Reinforcement Learning (DRL)

- **Model-Free Optimization:** DRL techniques offer a model-free alternative for navigating NP-hard wireless resource allocation problems by learning directly from environmental interactions.
- **Algorithm Selection:** Advanced frameworks frequently utilize Proximal Policy Optimization (PPO) for user scheduling tasks, while algorithms like Deep Deterministic Policy Gradient (DDPG) handle continuous power allocation.
- **Action Space Management:** To combat the curse of dimensionality in large networks, modern models integrate K-Nearest Neighbors (KNN) clustering to efficiently discretize continuous action spaces.

Enforcing Proportional Fairness in DRL

- **Fairness Formulations:** Proportional fairness can be mathematically embedded into the DRL objective by maximizing the logarithmic utility $\beta(z) = \log(z)$ of user data rates.
- **Dynamic Weighting:** Advanced DRL schedulers prevent cell-edge user starvation by dynamically applying scheduling weights based on historical throughput, actively punishing policies that leave users underserved.
- **Reward Penalization:** Agents can be trained with strict reward penalization, receiving zero rewards if their proposed continuous allocation violates predetermined Service Level Agreements.

My implementation

Shared Wireless System Model

The implementation studies an N-user interference channel with complex channel matrix H and power vector p.

$$\mathrm{SINR}_k(p, H) = \frac{|H_{k,k}|^2 p_k}{\sum_{j \neq k} |H_{k,j}|^2 p_j + \sigma^2},$$
$$R_k(p, H) = B \log_2 \left(1 + \mathrm{SINR}_k(p, H)\right),$$
$$\sum_{k=1}^{N} p_k \leq P_{\max}, \qquad p_k \geq 0.$$

- Implemented in `src/performance.py` and reused by evaluation scripts.
- Baselines solve different tradeoffs between sum-rate and fairness.
- Water filling is implemented in code but excluded here.

Dynamic CSI Transition

`WirelessAllocationEnv` and `DynamicChannelSimulator` model mobile fading with a first-order Gauss-Markov process:

$$H_{t+1} = \rho H_t + \sqrt{1-\rho^2} E_t, \qquad E_t \sim \mathcal{CN}(0,1).$$

The correlation coefficient follows Jakes' model:

$$\rho = J_0(2\pi f_d T_{\mathrm{TTI}}), \qquad f_d = \frac{v}{\lambda}.$$

- Carrier frequency: 2 GHz.
- Mobility setting in evaluations: 60 km/h.

Baseline 1: Fractional Programming

targets sum-rate maximization:

$$\max_{p \geq 0} \quad \sum_{k=1}^{N} \log\left(1 + \frac{G_{k,k} p_k}{\sum_{j \neq k} G_{k,j} p_j + \sigma^2}\right)$$
$$\text{s.t.} \quad \sum_{k=1}^{N} p_k \leq P_{\max}, \qquad G_{k,j} = |H_{k,j}|^2.$$

The code uses a quadratic-transform style auxiliary variable:

$$y_k = \frac{\sqrt{G_{k,k} p_k}}{\sum_{j \neq k} G_{k,j} p_j + \sigma^2 + G_{k,k} p_k}.$$

Power is updated by gradient ascent, clipped to nonnegative values, and projected back to the total power budget.

Baseline 2: Proportional-Fair WMMSE

weighted WMMSE with proportional-fair weights:

$$\alpha_k = \frac{1}{\overline{R}_k + \epsilon}.$$

With $v_k = \sqrt{p_k}$, each iteration updates receiver filters, MSE weights, and transmit amplitudes:

$$u_k = \frac{H_{k,k} v_k}{\sum_j |H_{k,j}|^2 v_j^2 + \sigma^2},$$

$$e_k = 1 - 2\Re(u_k H_{k,k} v_k) + |u_k|^2 \left(\sum_j |H_{k,j}|^2 v_j^2 + \sigma^2 \right),$$

$$w_k = \frac{\alpha_k}{\max(e_k, 10^{-9})}.$$

The transmit update uses KKT bisection to find μ so that $\sum_k v_k^2 \le P_{\max}$.

Baseline 3: Max-Min Fairness

`max_min_power_allocation` searches for the largest common SINR target γ that all users can satisfy:

$$\begin{aligned} \max_{p,\gamma} \quad & \gamma \\ \text{s.t.} \quad & \frac{G_{k,k}p_k}{\sum_{j\neq k} G_{k,j}p_j + \sigma^2} \geq \gamma, \quad \forall k, \\ & \sum_k p_k \leq P_{\max}, \qquad p_k \geq 0. \end{aligned}$$

For a fixed target γ, the required powers solve:

$$\left(I - \gamma D^{-1}F\right) p = \gamma D^{-1}\sigma^2 \mathbf{1}.$$

The implementation performs bisection over γ and keeps the highest feasible power vector.

Motivation: Why PPO?

- **Stability:** Standard Policy Gradients are sensitive to step sizes.
- **Simplicity:** PPO achieves the stability of TRPO but uses only first-order optimization.
- **Efficiency:** Better sample complexity than REINFORCE or A2C.

The Goal

Maximize performance while ensuring the new policy doesn't deviate too far from the old policy.

The Clipped Surrogate Objective

The core of PPO is the clipped objective function:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t) \right]$$

Where:

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio.
- $\hat{A}_t$ is the estimated advantage.
- ϵ is a hyperparameter (typically 0.1 or 0.2).

PPO: MDP Formulation

The DRL agent treats power allocation as an MDP $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$.

State

$$s_t = \left[\mathrm{vec}(|H_t|), \frac{\overline{R}_t}{10B} \right] \in \mathbb{R}^{N^2+N}.$$

Action

$$a_t \in [0,1]^N, \qquad p_t = P_{\max} a_t.$$

The state includes both instantaneous CSI and historical throughput so the policy can learn channel-aware and fairness-aware decisions.

PPO: Reward and Feasibility

The environment uses logarithmic utility to encode proportional fairness:

$$r_t = \sum_{k=1}^{N} \log(R_{k,t} + \epsilon) - 5 \max\left(0, \sum_k p_{k,t} - P_{\max}\right).$$

If the action exceeds the hardware budget, powers are projected back:

$$p_t \leftarrow p_t \frac{P_{\max}}{\sum_k p_{k,t}} \quad \text{when} \quad \sum_k p_{k,t} > P_{\max}.$$

Historical rates are updated with an exponential moving average:

$$\overline{R}_{k,t+1} = 0.8\overline{R}_{k,t} + 0.2R_{k,t}.$$

PPO: Policy Optimization

The training script uses Stable-Baselines3 PPO with an MLP policy.
PPO optimizes a clipped policy-gradient objective:

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right],$$

where

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t|s_t)}.$$

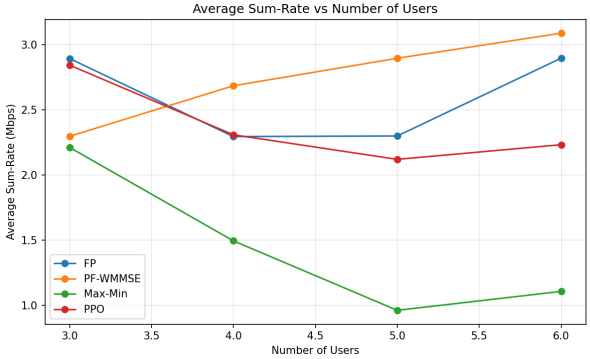
Clipping limits destructive policy updates while still allowing continuous power vectors.

PPO: Training Pipeline

Component	Implementation choice
Environment	`WirelessAllocationEnv(num_users=6, p_max=10)`
API validation	`check_env(env, warn=True)`
Policy	`PPO("MlpPolicy", env)`
Learning rate	3×10^{-4}
Rollout length	2048 steps
Batch size	64
Discount factor	$\gamma = 0.99$
Training horizon	200,000 timesteps

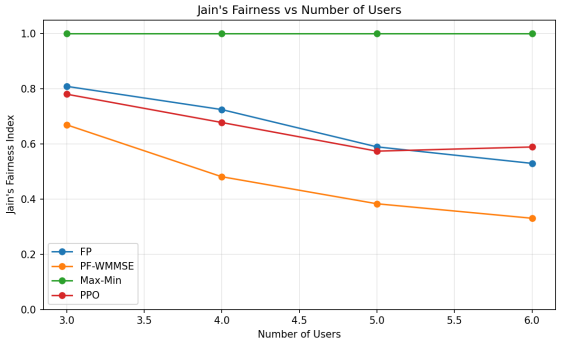
Separate saved models are used for $N \in \{3, 4, 5, 6\}$ in the scaling evaluations.

Results: Average Sum-Rate



- FP is the sum-rate-oriented upper reference and can sacrifice fairness.
- Max-min preserves equality but gives up total throughput.
- PPO is evaluated as a learned proportional-fair policy against PF-WMMSE.

Results: Jain Fairness

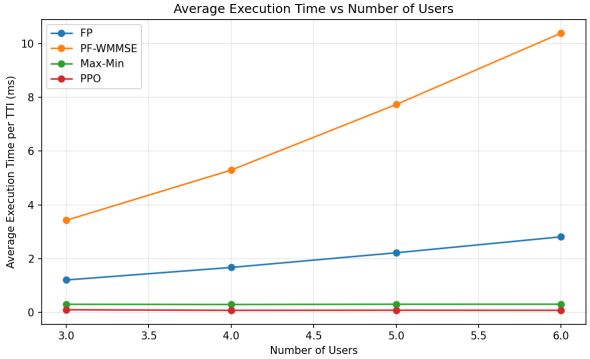


Jain's index is computed as

$$J(R) = \frac{\left(\sum_k R_k\right)^2}{N \sum_k R_k^2}.$$

PPO and PF-WMMSE are the relevant proportional-fair comparison because both include historical-rate feedback.

Results: Execution Time



- Classical baselines repeatedly solve iterative updates per TTI.
- PPO shifts most cost to offline training; online control is a deterministic forward pass.
- The implementation goal is not only high reward, but fast decisions under dynamic CSI.

Future Work

- The current PPO implementation is a proof of concept and has not been extensively tuned for performance.
- More extensive simulations with larger user counts and different mobility settings are needed to fully evaluate scalability and robustness.

Major Drawbacks

The cuurrent implementation has several limitations:

- The PPO agent is trained on a single mobility setting (60 km/h) and may not generalize well to other speeds or fading conditions.
- The PPO agent must train separate models for each user count, which limits scalability and adaptability in real-world deployments where user counts can fluctuate.

Meta-Learning for Adaptability

To address the scalability and generalization issues, future work could explore meta-learning approaches that enable the PPO agent to quickly adapt to new user counts and mobility settings with minimal additional training.

- Meta-learning algorithms like MAML could be used to train a single model that can fine-tune efficiently to new conditions.
- This would allow the agent to maintain high performance across a wider range of scenarios without needing separate models for each configuration.

MAML: Learning to Learn

- **Goal:** Train a model that can adapt to a new task with very few data points and a few gradient steps.
- **Key Idea:** Find an initialization θ such that the model is highly sensitive to the gradient updates of new tasks.
- **Agnostic:** Works with any model architecture that uses gradient descent.

The Difference

Standard Transfer Learning learns a good feature extractor; MAML learns an **optimal starting point** for optimization.

Bilevel Optimization Structure

For a task distribution $p(\mathcal{T})$, MAML updates parameters in two loops:

1. Inner Loop (Task-Specific):

$$\theta_i' = \theta - \alpha \nabla_\theta \mathcal{L}_{\mathcal{T}_i}(f_\theta) \tag{1}$$

2. Outer Loop (Meta-Update):

$$\theta \leftarrow \theta - \beta \nabla_\theta \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta_i'}) \tag{2}$$

Note: The outer loop requires computing gradients through gradients (Hessian-vector products).

Bibliography

- Z. Q. Luo and S. Zhang, "Dynamic spectrum management: Complexity and duality,"
- Meng, Fan & Chen, Peng & Wu, Lenan & Cheng, Julian. (2020). Power Allocation in Multi-User Cellular Networks: Deep Reinforcement Learning Approaches.
- S. He et al., "An Unsupervised Deep Unrolling Framework for Constrained Optimization Problems in Wireless Networks," *IEEE Trans. Wireless Commun.*, 2022.[2]
- H. H. S. Lopes et al., "Deep Deterministic Policy Gradient-Based Resource Allocation," *Sensors*, 2024.[4]
- A. Matemu et al., "Joint Scheduling and Power Allocation in O-RAN via PPO," *TechRxiv*, 2026.[4]
- M. Eisen and A. Ribeiro, "Optimal Wireless Resource Allocation With Random Edge Graph Neural Networks," *IEEE Trans. Signal Process.*, 2020.[6]
- H. Yang et al., "Knowledge-Driven Resource Allocation: A WMMSE Unrolled GNN Approach," *IEEE Internet of Things J.*, 2024